

캡스톤디자인 주간보고서

(9주차: 2026.04.30)

1. 보고자: 이동윤(202384026)

2. 작품명: Stringsense - AI기반 기타음원분리 및 자동채보 웹서비스

3. 작품 설명: 오디오 파일을 업로드하면 기타 연주 정보를 분석하여 기타 분리 음원, MIDI 파일, 기타 탭 초안을 제공하는 웹 서비스를 개발한다. 본 작품은 기타 채보 과정을 자동화하여 사용자가 수정 가능한 초안을 빠르게 얻을 수 있도록 하는 것을 목표로 한다.

4. 주요기능 및 작업 분담

- 오디오 파일 업로드 (MP3, WAV/Drag&Drop 지원)
- 기타 음원 분리
- 오디오 분석 기반 MIDI 생성
- 기타 탭 초안 생성
- 기본 웹 서비스 구조 설계
- 프로젝트 구조 및 기능 명세 정리

5. 개발 환경

- Frontend: React, VexFlow
- Backend: Python, FastAPI
- AI/ML: Demucs (Meta), Basic Pitch (spotify)
- 비동기: Celery, Redis
- 인프라: Docker
- 버전관리: Git, GitHub
- 개발 도구: VsCode

6. 진행사항

-3주차 작업 내용

프로젝트 주제를 자동 기타 패보 웹 서비스로 확정하고, 전체 기능 흐름을 정리하였다. 또한 현재 디렉터리에 프로젝트 프레임워크 문서, 기능 명세서, 디렉터리 구조 문서와 같은 기초 문서를 작성하여 작품의 목표, 주요기능, 범위, 구조를 정의하였다. 이를 통해 이후 구현 단계에서 어떤 기능을 우선적으로 개발할지 기준을 세웠다. 4주차에는 이 문서 내용을 바탕으로 실제 프로젝트 폴더 구조를 정리할 예정이다.

-4주차 작업 내용

3주차에 작성한 문서 내용을 바탕으로 현재 프로젝트 구조를 실제 디렉터리 형태로 정리하였다. 'frontend', 'backend', 'docs'를 중심으로 폴더를 구분하고, 문서 또한 원본 문서, 수정 문서, 계획서, 작업 기록으로 나누어 관리할 수 있도록 구성하였다. 또한 전체 개발 흐름을 다시 점검하면서 이후 구현 순서를 정리하였다. 다음 주에는 업로드 기능과 화면 구성을 중심으로 프론트엔드 시안을 구체화할 예정이다.

-5주차 작업 내용

사용자 관점에서 실제 서비스 처럼 보이는 프론트 화면 시안을 구현하였다. 'frontend' 폴더에 새 분석 시작 화면, 분석 진행 중 화면, 경과 화면, 탭 상세 화면을 구성하고, 각 화면이 서로 자연스럽게 연결되도록 정리하였다. 업로드 화면에서는 파일 선택, 드래그 앤 드롭, 샘플 불러오기, 분석 시작 기능이 보이도록 구성하였다. 처리 중 화면에서는 현재 분석이 진행되고 있다는 점을 직관적으로 보여주도록 로딩 상태와 단계 표시를 배치하였다.

결과 화면에서는 원본 음원, 기타 분리 음원, MIDI 다운로드, 탭 미리보기를 중심으로 배치하여 사용자가 결과를 바로 확인할 수 있게 하였다. 또한 탭 상세 화면에서는 생성된 기타 탭 초안을 크게 보고 저장하거나 결과 화면으로 돌아갈 수 있도록 구성하였다.

-6주차 작업 내용

-7주차 작업 내용

-9주차 작업 내용

-FastAPI 기반 백엔드 초기 구조를 구현하고 서버가 정상적으로 실행되는지 확인하였다. 'backend/app/main.py'에 FastAPI 앱 진입점을 작성하고 CORS 설정 및 프론트엔드 정적 파일 서빙을 구성하였다. 또한 Pydantic 기반 응답 스키마와 'POST/analyze','GET/files/{job_id}/{filename}' 등 핵심 라우트를 구현하였다.

- 오디오 처리 서비스로는 librosa 기반 전처리 모듈을 작성하여 업로드된 파일을 22050Hz mono로 정규화하고 60초 이내로 제한하도록 구성하였다. 음정 추출 서비스는 CQT(Constant-Q Transform)와 onset detection을 활용하여 음표 이벤트 목록을 반환하도록 구현하였다.

- 결과 생성 측면에서는 추출된 음표 이벤트를 pretty_midi를 통해 MIDI 파일로 변환하고, ASCII 형식의 기타 탭 텍스트로 렌더링하는 서비스를 구현하였다. 이후 전처리, 음정 추출, 결과 생성 단계를 하나로 연결하는 파이프라인 모듈을 작성하여 전체 분석 흐름이 하나의 API 호출로 처리

- 프론트엔드에서는 `app.js`에 실제 API 호출 로직을 추가하여 분석 시작 버튼을 클릭하면 `POST /analyze`로 파일을 전송하고 결과를 localStorage에 저장한 뒤 결과 화면으로 이동하도록 구현하였다. 또한 `result.html`과 `tab.html`에 id를 추가하여 JS 데이터 바인딩을 통해 실제 분석 결과가 화면에 표시되도록 연결하였다.

[illegible]